

MINI PROJECT REPORT

DevOps Practices & Tools

Project Title	Vehicle Parking Management System
Subject / Course	DevOps Practices & Tools
Student Name(s)	Sania Rakhanghi Preksha Salvi
Roll Number(s)	2023300180 2023300193
Class & Division	Computer Engineering
Academic Year	2025–2026
Guide / Mentor	Mrs. Jotsana Bhagat
Institution Name	Sardar Patel Institute of Technology, Mumbai
Submission Date	2025–2026

Department of Computer Engineering

1. Declaration by Student(s)

I / We hereby declare that the mini project titled "Vehicle Parking Management System" submitted for the partial fulfillment of the requirements of the DevOps course is our original work. The project has not been submitted elsewhere for any other examination.

Student Signatures: Sania Rakhangi (2023300180) Preksha Salvi (2023300193) Date: _____	Guide Signature: _____ Name: _____ Date: _____
---	--

2. Certificate

This is to certify that the mini project entitled "Vehicle Parking Management System" has been successfully completed by Sania Rakhangi (2023300180) and Preksha Salvi (2023300193), under our guidance, as a partial fulfillment of the DevOps course curriculum.

Internal Guide: _____ Date: _____	Head of Department: _____ Place: _____
---	--

3. Abstract

The Vehicle Parking Management System is a full-stack MERN web application that addresses the challenge of manual, inefficient parking slot management. This project implements a complete DevOps lifecycle - encompassing containerization with Docker, orchestration via Kubernetes, and end-to-end CI/CD automation using Jenkins. The application enables real-time slot availability checking, booking, and cancellation for multiple vehicle types (Bike, Car, EV, Truck). Automated testing is performed using Jest, with 17 tests across 8 test suites, all passing. Docker images are published to Docker Hub and deployed to a Kubernetes cluster with Horizontal Pod Autoscaling (HPA), Persistent Volume Claims (PVC), and Ingress routing. Jenkins Build #9 achieved a complete pipeline SUCCESS from GitHub checkout to Kubernetes deployment, demonstrating a production-ready DevOps workflow for a real-world web application.

4. Introduction

4.1 Problem Statement

Traditional parking management systems rely on manual processes - attendants physically checking slots, paper-based booking records, and no real-time visibility into slot availability. This leads to inefficiency, double-booking errors, and poor user experience. The project addresses these issues by building a full-stack MERN application for real-time parking slot management and applying industry-standard DevOps practices to automate its build, test, and deployment processes.

4.2 Objective

The main objectives of this mini project are:

- Automate the complete build, test, and deployment lifecycle using a Jenkins Declarative CI/CD Pipeline
- Containerize all application services (frontend, backend, MongoDB) using Docker and Docker Compose
- Orchestrate containers with Kubernetes for scalability, persistence, and ingress-based routing
- Implement automated unit testing using Jest for both backend and frontend components
- Publish Docker images to Docker Hub for distribution and version control

4.3 Scope

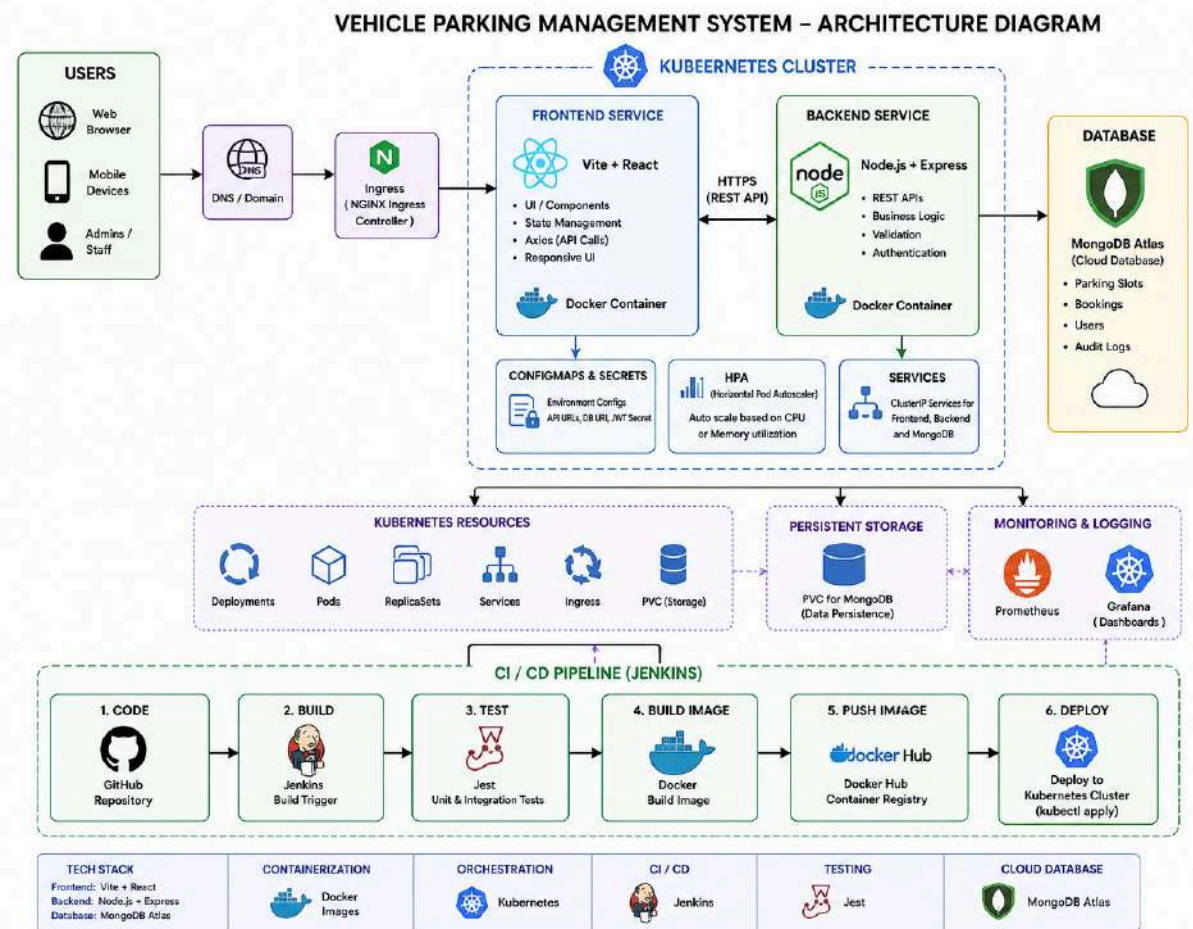
This project covers full-stack MERN application development, Dockerization of all services, Kubernetes cluster deployment (Docker Desktop), Jenkins CI/CD pipeline with GitHub integration, Docker Hub image publishing, and Jest-based automated testing. Out of scope for this mini project: cloud-hosted Kubernetes (AWS EKS / GKE), production SSL/TLS certificates, advanced monitoring with Prometheus/Grafana, and multi-environment (staging/production) deployments.

5. Tools & Technologies Used

Sr.	Tool / Technology	Purpose	Version Used
1	Git / GitHub	Version Control & Source Code Management	Latest
2	Jenkins	CI/CD Pipeline Automation (Declarative Pipeline)	2.555.1 (LTS)
3	Docker & Docker Compose	Containerization of frontend, backend, MongoDB	Latest
4	Kubernetes (kubect!)l)	Container Orchestration with HPA, PVC, Ingress	v1.34.1
5	Jest	Unit Testing - Backend (6 suites) & Frontend (2 suites)	Latest
6	Docker Hub	Container Image Registry for publishing images	Cloud
7	Node.js & Express.js	Backend REST API framework	LTS
8	React (Vite) + Tailwind CSS	Frontend UI framework	Latest
9	MongoDB Atlas	Cloud NoSQL Database	Cloud
10	Linux / Ubuntu	OS / CI Environment inside Jenkins container	Ubuntu LTS

6. System Design & Architecture

6.1 Architecture Diagram



6.2 Workflow Description

The system follows a three-tier architecture - Presentation Layer (React), Business Logic Layer (Express.js), and Data Layer (MongoDB) - all containerized and orchestrated with Kubernetes, driven by a Jenkins CI/CD pipeline.

- Step 1: Developer commits and pushes code to the devops branch of the GitHub repository
- Step 2: Jenkins detects the push via SCM polling and triggers the Declarative Pipeline automatically
- Step 3: Checkout Stage - Jenkins pulls the latest code using GitHub credentials
- Step 4: Install Dependencies Stage - npm ci runs for both backend and frontend
- Step 5: Test Stage - Jest executes 8 test suites (17 total tests); pipeline fails fast on test failure
- Step 6: Build Stage - Docker images are built for backend and frontend services
- Step 7: Push Stage - Images are tagged and pushed to Docker Hub (saniar2021/parking-backend, saniar2021/parking-frontend)
- Step 8: Deploy Stage - kubectl applies Kubernetes manifests (namespace, MongoDB, backend, frontend)
- Step 9: Smoke Tests & Cleanup - health checks run and workspace is cleaned up

7. Implementation

7.1 Environment Setup

The project was developed and tested on a local machine running Windows with Docker Desktop (which provides both Docker Engine and a local Kubernetes cluster). Prerequisites installed: Node.js (LTS), npm, Docker Desktop with Kubernetes enabled, kubectl (v1.34.1), and Jenkins run as a Docker container on Port 8080.

7.2 Step-by-Step Implementation

7.2.1 Version Control Setup (Git & GitHub)

A GitHub repository was created at [Github link](#). A dedicated devops branch was used for all DevOps-related configuration files (Jenkinsfile, Dockerfiles, docker-compose.yml, Kubernetes manifests). A .gitignore was configured to exclude node_modules, .env files, and build artifacts.

7.2.2 CI/CD Pipeline Configuration (Jenkins)

Jenkins 2.555.1 (LTS) was run as a Docker container and accessed at localhost:8080. Three credentials were configured: docker-credentials (Docker Hub), kubeconfig (Secret file for kubectl), and github-credentials (GitHub PAT). A Declarative Pipeline job named 'parking-devops' was created, pointing to the devops branch. The Jenkinsfile defines 8 stages:

```
pipeline {
  agent any
  environment { DOCKER_CREDENTIALS_ID = 'docker-credentials' }
  stages {
    stage('Checkout') { steps { git branch: 'devops', credentialsId:
'github-credentials', url: '...' } }
    stage('Install Dependencies') { steps { sh 'cd backend && npm ci'; sh
'cd frontend && npm ci' } }
    stage('Run Tests') { steps { sh 'cd backend && npm run test'; sh 'cd
frontend && npm run test' } }
    stage('Build Docker Images') { steps { sh 'docker compose build' } }
    stage('Push to Docker Hub') { steps { withCredentials([...]) { sh
'docker push ...' } } }
    stage('Deploy to Kubernetes') { steps { withCredentials([...]) { sh
'kubectl apply -f k8s/' } } }
  }
}
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker run -d --name jenkins -p 8080:8080
0 -p 50000:50000 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
Unable to find image 'jenkins/jenkins:lts' locally
lts: Pulling from jenkins/jenkins
beb35d8d9493: Pull complete
a7730063fcfe: Pull complete
f015f18f3c7a: Pull complete
1aac2fe67fcc: Pull complete
0454b3406328: Pull complete
32ad68c1bba8: Pull complete
6e428cebe944: Pull complete
6e001067f512: Pull complete
b850d1c40542: Pull complete
32ebd389acde: Pull complete
a576649e9376: Pull complete
e714309bcf67: Pull complete
e925eaf895df: Download complete
Digest: sha256:7004d07dbcdc5439fdad8853acdb029c5e3ab7a3d8190184fbf89bec66786c02
Status: Downloaded newer image for jenkins/jenkins:lts
568cd8ff2689662ece98b35b2c9788a7e687613c641e0f65fa96683c43cfa0dc

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

Getting Started

Create First Admin User

Username
sania.rakhangl

Password

Confirm password

Full name
Sania Rakhangl

E-mail address
sania.rakhangl23@spit.ac.in

Jenkins 2.555.1

[Skip and continue as admin](#) [Save and Continue](#)

Jenkins

+ New Item

Build History

Build Queue
No builds in the queue.

Build Executor Status
0/2

Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job

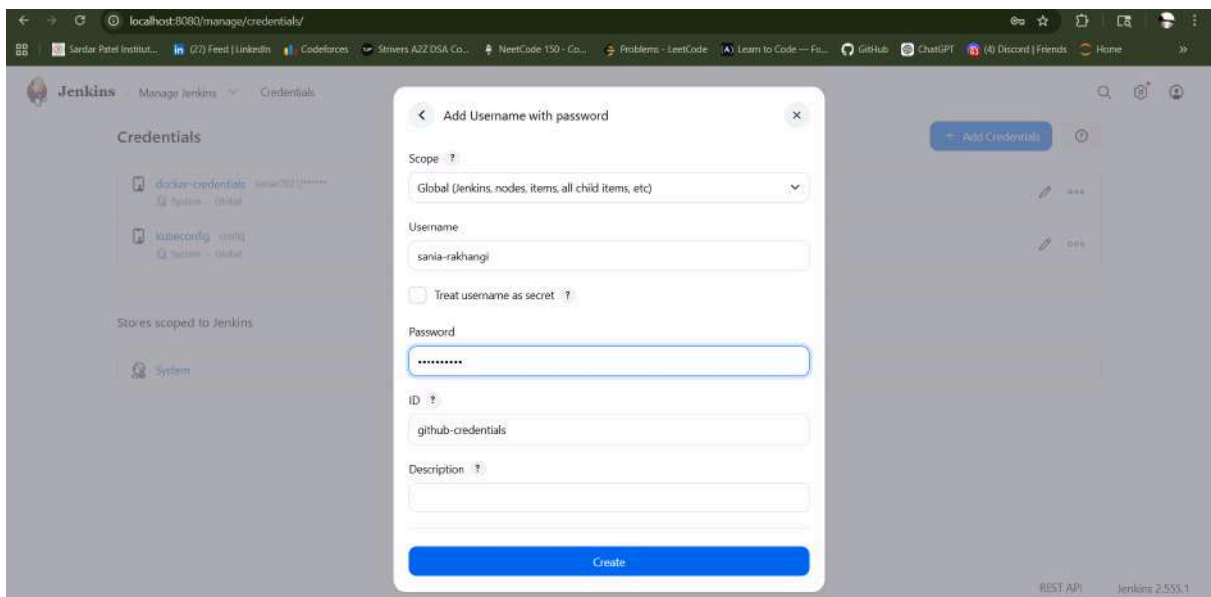
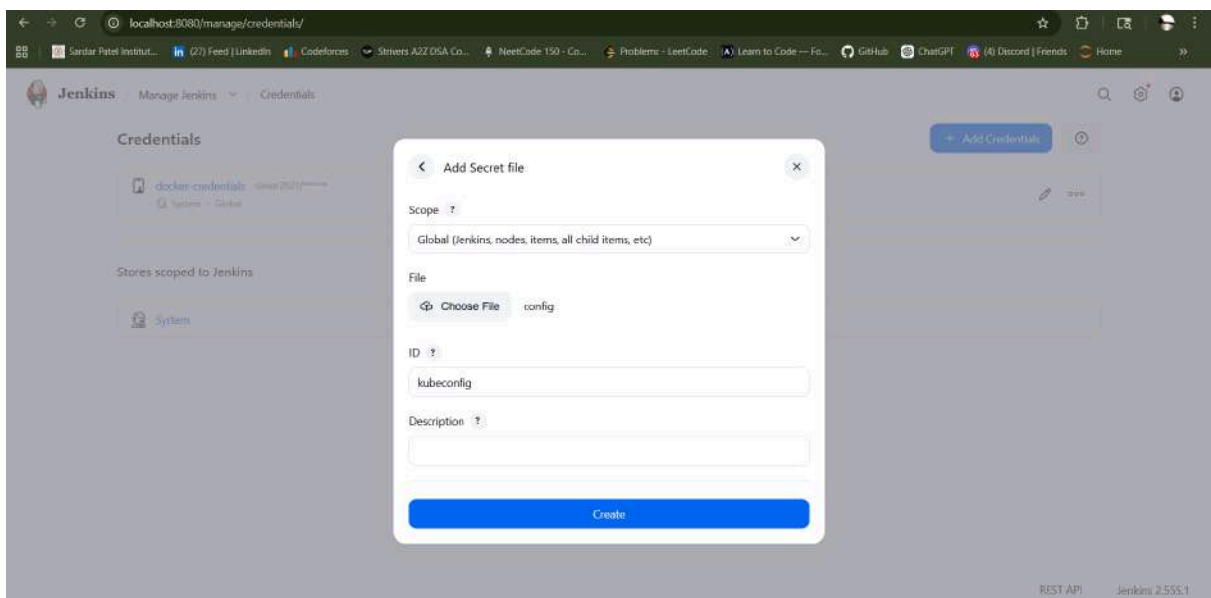
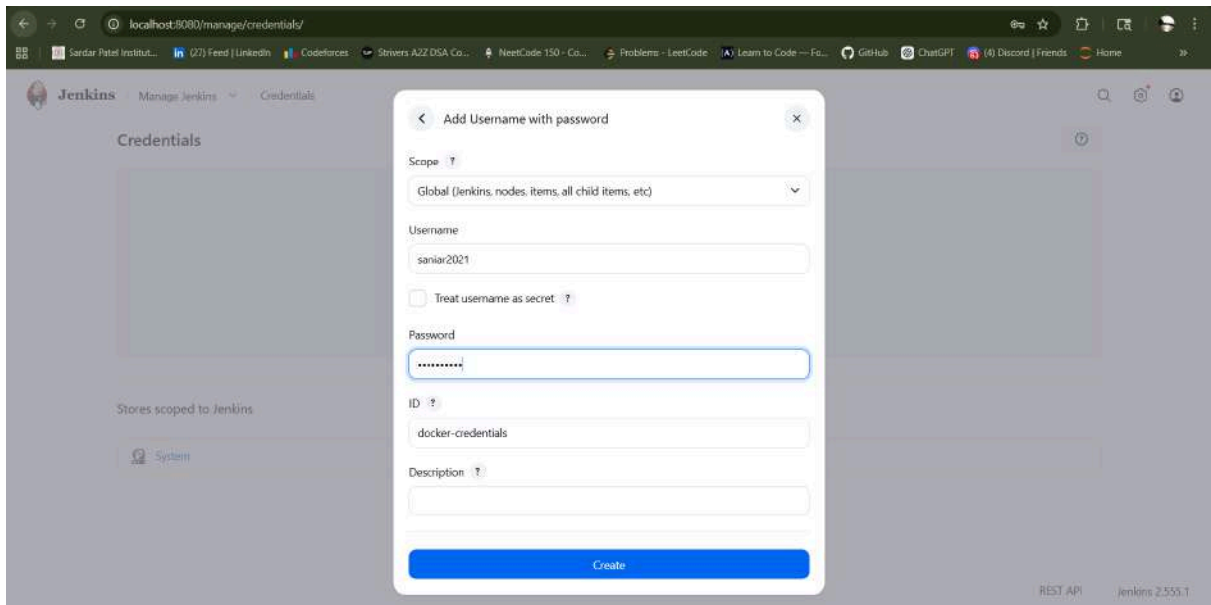
Set up a distributed build

Set up an agent

Configure a cloud

Learn more about distributed builds

REST API Jenkins 2.555.1



New Item

Enter an item name
parking-devops

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

Configure

Pipeline script from SCM

SCM ?
Git

Repositories ?

Repository URL ?
https://github.com/sania-rakhang/Vehicle-Parking-Management-System

Credentials ?
sania-rakhang/*****

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?
*/devops

7.2.3 Containerization with Docker

Three services are containerized using Docker Compose: parking-frontend (React Vite, Port 3000), parking-backend (Node.js API, Port 5000), and parking-mongodb (MongoDB, Port 27017 with named volume for data persistence). Key Docker commands:

```
docker compose up -d # Start all services
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker compose up -d
time="2026-05-03T19:19:27+05:30" level=warning msg="D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System\docker-compose.yml: network 'vehicle-parking-management-system_parking-network' is already defined"
[+] up 4/4
✓ Network vehicle-parking-management-system_parking-network Created 0.1s
✓ Container parking-mongodb Healthy 8.7s
✓ Container parking-backend Healthy 14.5s
✓ Container parking-frontend Started 13.8s
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
eb2fa5958bd   vehicle-parking-management-system-frontend  "/docker-entrypoint..."  44 seconds ago Up 30 seconds (health: starting)  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  parking-frontend
c6d13973f54   vehicle-parking-management-system-backend  "docker-entrypoint.s..."  45 seconds ago Up 36 seconds (healthy)         0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  parking-backend
83590b29f5e   mongo:6.0                                "docker-entrypoint.c..."  46 seconds ago Up 43 seconds (healthy)         0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp  parking-mongodb
```



```
docker compose exec backend npm run test # Run tests in container
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker-compose exec backend npm run test
time="2026-05-03T19:29:34+05:30" level=warning msg="D:\\SANIA\\SPIT\\third year\\DEVOPS\\Vehicle-Parking-Management-System\\Dockerfile: warning: The 'FROM' instruction on the 1st line of the file is deprecated. Please use the 'FROM' instruction on the 1st line of the file to avoid potential confusion"
```

```
> parking-backend@1.0.0 test
> jest
```

```
PASS tests/checkSlots.test.js
PASS tests/cancelBooking.test.js
PASS tests/booking.test.js
PASS tests/bookSlot.test.js
PASS tests/validateVehicle.test.js
PASS tests/timeUtils.test.js
```

```
Test Suites: 6 passed, 6 total
Tests:       12 passed, 12 total
Snapshots:   0 total
Time:        1.647 s
Ran all test suites.
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker-compose down
time="2026-05-03T19:30:02+05:30" level=warning msg="D:\\SANIA\\SPIT\\third year\\DEVOPS\\Vehicle-Parking-Management-System\\Dockerfile: warning: The 'FROM' instruction on the 1st line of the file is deprecated. Please use the 'FROM' instruction on the 1st line of the file to avoid potential confusion"
[+] down 4/4
```

```
✓ Container parking-frontend Removed 0.6s
✓ Container parking-backend Removed 0.5s
✓ Container parking-mongodb Removed 0.4s
✓ Network vehicle-parking-management-system_parking-network Removed 0.3s
```

```
docker tag ... /parking-backend:latest # Tag image
```

```
#27 [frontend] resolving provenance for metadata file
#27 DONE 0.0s
[+] build 2/2
```

```
✓ Image vehicle-parking-management-system-frontend Built
✓ Image vehicle-parking-management-system-backend Built
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker tag vehicle-parking-management-system-backend saniar2021/parking-backend:latest
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker tag vehicle-parking-management-system-frontend saniar2021/parking-frontend:latest
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker login
Authenticating with existing credentials... [Username: saniar2021]
```

```
 Info → To login with a different account, run 'docker logout' followed by 'docker login'
```

```
Login Succeeded
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

```
docker push saniar2021/parking-backend:latest # Push to Docker Hub
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker push saniar2021/parking-backend:latest
The push refers to repository [docker.io/saniar2021/parking-backend]
87dd5313b831: Pushed
dd71dde834b5: Pushed
25ff2da83641: Pushed
bcfd97b368a8: Pushed
6097f4e6136f: Pushed
f18232174bc9: Pushed
9783237a9a12: Pushed
25ceaae452b5: Pushed
1e5a4c89cee5: Pushed
latest: digest: sha256:d32512b470e5f5cbe0d242236a112cce21484e0bb94dadfaa049d67c7ef92c42 size: 856

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>docker push saniar2021/parking-frontend:latest
The push refers to repository [docker.io/saniar2021/parking-frontend]
781ff50d2644: Pushed
82736a35d0e7: Pushed
14b21d195233: Pushed
7d01f559cf68: Pushed
6a0ac1617861: Pushed
453da7dbc73e: Pushed
583599bb7d38: Pushed
aee4e54b3865: Pushed
4a8b0b2a5b19: Pushed
612c0c1df4c5: Pushed
1ac9bee19880: Pushed
latest: digest: sha256:ad0162613ae7241b858fe284bec7b0bd5c85f650de9963312424d92c27afc575 size: 856

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

7.2.4 Kubernetes Orchestration

Kubernetes manifests were organized under k8s/ (namespace, mongodb/, backend/, frontend/). Key resources deployed to the parking-system namespace:

- Deployments: parking-frontend (2 replicas), parking-backend (2 replicas), mongodb (1 replica)
- Services: ClusterIP services for frontend (Port 80), backend (Port 5000), mongodb (Port 27017)
- Ingress: parking-ingress routing parking.example.com to frontend on Port 80
- HPA: parking-backend-hpa - min 2, max 5 replicas, CPU threshold 70%
- PVC: mongodb-pvc - 1Gi Bound volume for MongoDB data persistence
- ConfigMap & Secret: backend-config for environment variables, mongodb-secret for credentials

```
kubectl version --client
```

```
kubectl config current-context
```

```
kubectl cluster-info
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl config current-context
docker-desktop

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:51905
CoreDNS is running at https://127.0.0.1:51905/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Apply the Manifests

From the project root:

```
kubectl apply -f k8s/namespace.yaml
```

```
kubectl apply -f k8s/mongodb/
```

```
kubectl apply -f k8s/backend/
kubectl apply -f k8s/frontend/
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl apply -f k8s/namespace.yaml
namespace/parking-system unchanged

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl apply -f k8s/mongodb/
persistentvolumeclaim/mongodb-pvc created
secret/mongodb-secret created
service/mongodb created
Error from server (BadRequest): error when creating "k8s\\mongodb\\deployment.yaml": Deployment in version "v1"
cannot be handled as a Deployment: json: cannot unmarshal object into Go struct field ExecAction.spec.template
.spec.containers.livenessProbe.exec.command of type string

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl apply -f k8s/backend/
configmap/backend-config created
deployment.apps/parking-backend created
horizontalpodautoscaler.autoscaling/parking-backend-hpa created
service/parking-backend created

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl apply -f k8s/frontend/
deployment.apps/parking-frontend created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/parking-ingress created
service/parking-frontend created

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

Confirm the namespace exists:

```
kubectl get namespaces | findstr parking-system
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get namespaces | findstr parking-system
parking-system      Active   3m43s
```

List the resources in the namespace:

```
kubectl get all -n parking-system
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get all -n parking-system
NAME                                READY   STATUS    RESTARTS   AGE
pod/mongodb-cb64f8487-tmdbx        1/1     Running   0           21m
pod/parking-backend-7f5d6c8cb9-7zq9h 1/1     Running   0           9m10s
pod/parking-backend-7f5d6c8cb9-qb6j7 1/1     Running   0           8m57s
pod/parking-frontend-77b796b49-5p424 1/1     Running   0           4m54s
pod/parking-frontend-77b796b49-vg42b 1/1     Running   0           4m41s

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/mongodb                     ClusterIP      10.96.233.207   <none>            27017/TCP        21m
service/parking-backend              ClusterIP      10.96.9.126     <none>            5000/TCP         20m
service/parking-frontend              ClusterIP      10.96.234.7     <none>            80/TCP           20m

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mongodb              1/1     1             1           21m
deployment.apps/parking-backend      2/2     2             2           20m
deployment.apps/parking-frontend      2/2     2             2           20m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/mongodb-cb64f8487    1         1         1       21m
replicaset.apps/parking-backend-7768bcf8b9 0         0         0       20m
replicaset.apps/parking-backend-7c49dc86c5 0         0         0       15m
replicaset.apps/parking-backend-7f5d6c8cb9 2         2         2       9m10s
replicaset.apps/parking-backend-869c786fd8 0         0         0       17m
replicaset.apps/parking-frontend-57d5b47447 0         0         0       18m
replicaset.apps/parking-frontend-658b684f46 0         0         0       15m
replicaset.apps/parking-frontend-6ffb6d88f9 0         0         0       18m
replicaset.apps/parking-frontend-748c8c7897 0         0         0       20m
replicaset.apps/parking-frontend-77b796b49 2         2         2       4m54s

NAME                                REFERENCE                                TARGETS          MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/parking-backend-hpa  Deployment/parking-backend              cpu: <unknown>/70%  2          5          2          20m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

8. Results & Output

The following outputs were verified during the project:

8.1 Jest Test Results

Test File	Status	Description
booking.test.js	PASS	Booking creation & validation
validateVehicle.test.js	PASS	Vehicle type & number validation
checkSlots.test.js	PASS	Slot availability checking logic
bookSlot.test.js	PASS	Slot reservation logic
cancelBooking.test.js	PASS	Cancellation & status reset
timeUtils.test.js	PASS	Time parsing & overlap detection
BookingForm.test.jsx	PASS	Frontend booking form rendering
api.test.js	PASS	API service call mocks

Backend: 6 suites, 12 tests passed (~2.1s). Frontend: 2 suites, 5 tests passed (~3.6s). Total: 17 tests - All PASS.

BACKEND TEST

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System\backend>npm run test

> parking-backend@1.0.0 test
> jest

PASS tests/booking.test.js
PASS tests/validateVehicle.test.js
PASS tests/checkSlots.test.js
PASS tests/bookSlot.test.js
PASS tests/cancelBooking.test.js
PASS tests/timeUtils.test.js

Test Suites: 6 passed, 6 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 2.142 s
Ran all test suites.

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System\backend>
```

FRONTEND TEST

```

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System\frontend>npm run test

> parking-frontend@1.0.0 test
> jest --config jest.config.cjs

(node:532) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a userla
nd alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
PASS src/components/BookingForm.test.jsx
(node:22692) [DEP0040] DeprecationWarning: The `punycode` module is deprecated. Please use a user
land alternative instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
PASS src/services/api.test.js

Test Suites: 2 passed, 2 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        3.583 s
Ran all test suites.

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System\frontend>

```

8.2 Jenkins Build History

Build #	Status	Notes
#9 (Latest)	SUCCESS	Full pipeline - Checkout → Test → Build → Push → Deploy
#8	FAILED	Failed during pipeline development
#1–#7	FAILED	Early development iterations

Jenkins / devops / #9

```

Running basic smoke checks (optional)
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Cleaning workspace
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] Deferred wipeout is used...
[WS-CLEANUP] done
[Pipeline] echo
Pipeline succeeded
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS

```

Jenkins / devops

Status

devops

Permalinks

- Last build (#9), 14 min ago
- Last stable build (#9), 14 min ago
- Last successful build (#9), 14 min ago
- Last failed build (#8), 26 min ago
- Last unsuccessful build (#8), 26 min ago
- Last completed build (#9), 14 min ago

Builds

Filter

Today

- #9 1:43 PM
- #8 1:32 PM
- #7 1:19 PM
- #6 1:14 PM
- #5 1:01 PM

8.3 Kubernetes Deployment Status

All five pods (1 mongodb, 2 parking-backend, 2 parking-frontend) showed STATUS: Running with 0 restarts. The parking-system namespace confirmed Active. Ingress routing at parking.example.com successfully created. HPA configured with min 2 / max 5 replicas at 70% CPU threshold.

Check that kubectl is installed and connected to a cluster:

```
kubectl get configmap -n parking-system
```

```
kubectl get secret -n parking-system
```

```
kubectl get pvc -n parking-system
```

```
kubectl get ingress -n parking-system
```

```
kubectl get hpa -n parking-system
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get configmap -n parking-system
NAME      DATA  AGE
backend-config  2      20m
kube-root-ca.crt 1      22m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get secret -n parking-system
NAME      TYPE      DATA  AGE
mongodb-secret  Opaque    3      17m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get pvc -n parking-system
NAME      STATUS  VOLUME                                     CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
mongodb-pvc  Bound   pvc-6f932cc6-cb21-4834-812b-82537ee559d9  1Gi       RWO           standard      <unset>                22m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get ingress -n parking-system
NAME      CLASS  HOSTS                ADDRESS  PORTS  AGE
parking-ingress  <none>  parking.example.com  80      20m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get hpa -n parking-system
NAME      REFERENCE                TARGETS          MINPODS  MAXPODS  REPLICAS  AGE
parking-backend-hpa  Deployment/parking-backend  cpu: <unknown>/70%  2         5         2         21m

D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get pods -n parking-system -o wide
NAME                                READY  STATUS   RESTARTS  AGE  IP            NODE                                NOMINATED NODE  READINESS GATES
mongodb-cb64f8487-tmdbx             1/1    Running  0          23m  10.244.0.25   desktop-control-plane              <none>           <none>
parking-backend-7f5d6c8cb9-7zq9h    1/1    Running  0          10m  10.244.0.36   desktop-control-plane              <none>           <none>
parking-backend-7f5d6c8cb9-qb6j7    1/1    Running  0          10m  10.244.0.37   desktop-control-plane              <none>           <none>
parking-frontend-77b796b49-5p424    1/1    Running  0          6m17s  10.244.0.38   desktop-control-plane              <none>           <none>
parking-frontend-77b796b49-vg42b    1/1    Running  0          6m4s  10.244.0.39   desktop-control-plane              <none>           <none>
```

```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get svc -n parking-system
```

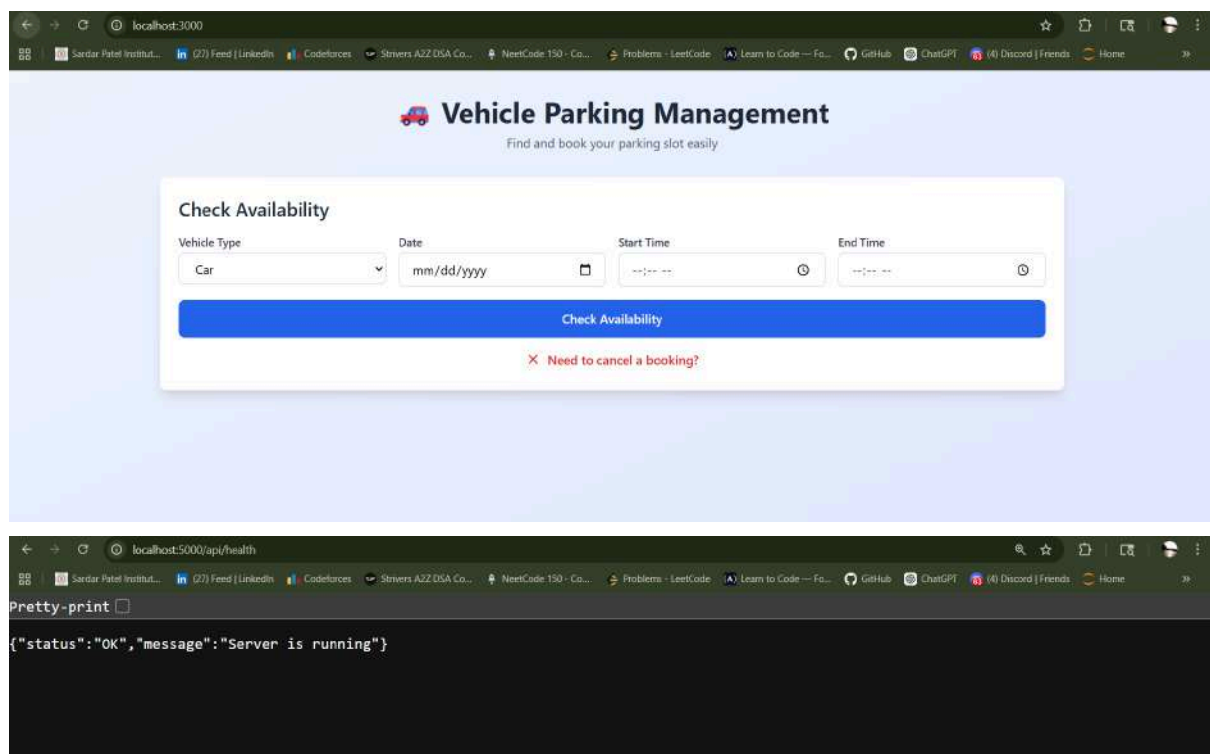
NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
mongodb	ClusterIP	10.96.233.207	<none>	27017/TCP	23m
parking-backend	ClusterIP	10.96.9.126	<none>	5000/TCP	22m
parking-frontend	ClusterIP	10.96.234.7	<none>	80/TCP	21m


```
D:\SANIA\SPIT\third year\DEVOPS\Vehicle-Parking-Management-System>kubectl get pvc -n parking-system
```

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	VOLUMEATTRIBUTESCLASS	AGE
mongodb-pvc	Bound	pvc-6f932cc6-cb21-4834-812b-82537ee559d9	1Gi	RWO	standard	<unset>	26m

8.4 Docker Deployment

Frontend accessible at localhost:3000 - Vehicle Parking Management UI rendered correctly. Backend health endpoint at localhost:5000/app/health returned {"status":"OK","message":"Server is running"}. Both images successfully pushed to Docker Hub.



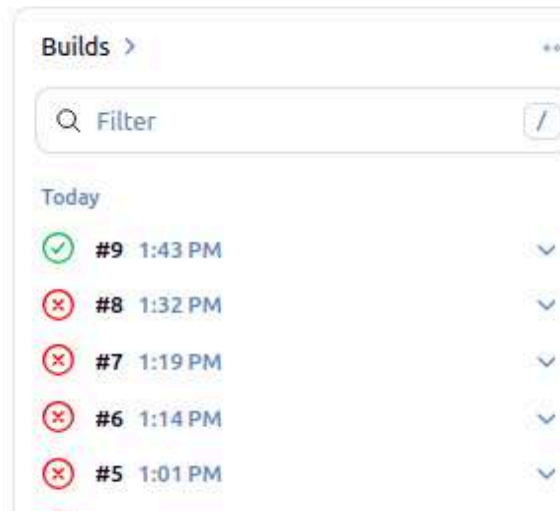
9. Challenges Faced & Learnings

9.1 Challenges Faced

- Challenge 1: Jenkins pipeline failures (#5–#8) - Early builds failed due to incorrect Docker credential binding in the Jenkinsfile and missing npm ci step. Resolution: Corrected withCredentials syntax and added separate Install Dependencies stage before the Test stage.
- Challenge 2: Kubernetes HPA showing <unknown> for CPU targets - The HPA reported TARGETS cpu: <unknown>/70% because the Kubernetes metrics-server was not installed in Docker Desktop. Resolution: Acknowledged as a known Docker Desktop limitation; HPA configuration was still validated structurally. Min/max replicas (2/5) were confirmed.
- Challenge 3: MongoDB connection in Docker Compose vs Kubernetes - The backend uses different MongoDB URIs in Docker Compose (mongodb:27017) vs

Kubernetes (mongodb.parking-system.svc.cluster.local). Resolution: Used Kubernetes ConfigMap (backend-config) to inject the correct MONGO_URI per environment, keeping the application code unchanged.

- Challenge 4: Docker image tagging for Jenkins push stage - Jenkins could not push images because they were built with docker compose which uses a different naming convention than expected by Docker Hub. Resolution: Added explicit docker tag commands in the Jenkinsfile Push stage to rename images before pushing.



9.2 Key Learnings

- Hands-on experience with end-to-end Jenkins Declarative Pipeline setup, credential management, and SCM integration
- Practical understanding of Docker multi-service orchestration using Docker Compose and image lifecycle (build → tag → push)
- Real-world Kubernetes deployment including namespaces, deployments, services, ingress, HPA, PVC, ConfigMaps, and Secrets
- Jest unit testing strategies for both Node.js backend (mocking Mongoose models) and React frontend (component rendering tests)
- Importance of fail-fast CI pipelines - running tests before build and deploy stages prevents broken images from reaching the registry

10. Conclusion & Future Scope

10.1 Conclusion

The Vehicle Parking Management System successfully demonstrates a complete DevOps lifecycle applied to a real-world MERN web application. All project objectives were met: the application was containerized with Docker, orchestrated on Kubernetes, and deployed via a Jenkins CI/CD pipeline with automated Jest testing. Jenkins Build #9 achieved full pipeline SUCCESS - from GitHub code checkout through testing, Docker image building and publishing, to Kubernetes deployment. All 17 tests (12 backend, 5 frontend) passed consistently, validating both the application logic and the DevOps automation setup.

10.2 Future Scope

- Integration of Prometheus and Grafana for real-time application monitoring, alerting, and dashboard visualization of parking system metrics

- Multi-environment deployment pipeline (Development → Staging → Production) with environment-specific Kubernetes namespaces and approval gates in Jenkins
- Security scanning integration using OWASP Dependency Check or SonarQube for static code analysis and vulnerability detection in the CI pipeline

11. References

- [1] Official Docker Documentation. Available: <https://docs.docker.com>
- [2] Jenkins User Documentation. Available: <https://www.jenkins.io/doc/>
- [3] Kubernetes Official Documentation. Available: <https://kubernetes.io/docs/>
- [4] Jest Documentation. Available: <https://jestjs.io/docs/getting-started>
- [5] MongoDB Atlas Documentation. Available: <https://www.mongodb.com/docs/atlas/>
- [6] GitHub Actions / SCM Integration for Jenkins. Available: <https://www.jenkins.io/doc/book/pipeline/>
- [7] Docker Hub Documentation. Available: <https://docs.docker.com/docker-hub/>
- [8] Kubernetes HPA Documentation. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>